TCP Congestion Control

Data Network — Transport Layer

2026-04-16

Table of contents

1 (Overview) 2
- 1.1 “Router” ? . . . 2
- 1.2 . . . 2

2 cwnd 3
- 2.1 cwnd ? . . . 3
- 2.2 cwnd ? . . . 3
- 2.3 cwnd — cwnd ? . . . 3

3 (Costs of Congestion) 4
- 3.1 Scenario 1: (,) . . . 4
- 3.2 Scenario 2: + ( ) . . . 5
- 3.3 Scenario 3: ( ) . . . 5

4 TCP 6
- 4.1 Slow Start (SS) . . . 6
- 4.2 Congestion Avoidance (CA) . . . 8
- 4.3 Fast Retransmit & Fast Recovery (FR) . . . 10

5 TCP Tahoe vs. TCP Reno 10

6 TCP FSM 12

7 TCP Throughput 13
- 7.1 (AIMD) . . . 13
- 7.2 Mathis Formula ( ) . . . 14
- 7.3 Long Fat Pipe . . . 14

8 TCP (Fairness) 15
- 8.1 AIMD . . . 15
- 8.2 . . . 18

9 Explicit Congestion Notification (ECN) 18
- 9.1 . . . 18

10 **18**
10.1 18
10.2 cwnd 19
10.3 19

11 **21**

1 (Overview)

TCP Congestion Control (congestion) , **- - (end-to-end)** . IP , TCP **(loss event)** (implicit) .

> **Note**
>
> **Flow Control vs. Congestion Control**
>
> | Flow Control | Congestion Control |
> |---|---|
> | `rwnd` (Receive Window) Sender Receiver | (Timeout / 3 Dup ACK) |

1.1 “Router” ?

“router ”, “router ” . Switch , router ?

( )

Switch L2( ) MAC , Router L3( ) IP . TCP Transport Layer , TCP “ ” IP , **router** . TCP router .

Switch . TCP “IP ” router .

IP layer ( )

IP best-effort , TCP (ECN). TCP router , **/ RTT** . “end-end congestion control (no network assistance)” .

> **Tip**
>
> . Router **L3 IP** , IP TCP .

1.2

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\underbrace{cwnd}_{\text{congestion window}}, \underbrace{rwnd}_{\text{flow control window}})$$

- **`cwnd` (Congestion Window)**: unACKed . .

- **`ssthresh` (Slow Start Threshold)**: Slow Start Congestion Avoidance .
- :

$$\text{rate} \approx \frac{cwnd}{RTT} \quad \text{[bytes/sec]}$$

---

2 cwnd

2.1 cwnd ?

`Congwin` = **Congestion Window (`cwnd`)** . " " , .

2.2 cwnd ?

TCP sender (invariant) :

$$\text{LastByteSent} - \text{LastByteAcked} \leq cwnd$$

, **" (in-flight)" (unACKed)** cwnd . cwnd ACK .
.

$$\approx \frac{cwnd}{RTT} \text{ [bytes/sec]}$$

2.3 cwnd — cwnd ?

4 :

`send_base`	Already ACKed ( )
`send_base` ~ `nextseqnum`	**Sent, not yet ACKed** (ACK)
`nextseqnum` ~ cwnd	**Usable, not yet sent** ( )
cwnd	Not usable ( )

cwnd **" (upper bound)"** , . (usable, not yet sent) Application
, .

> **Note**
>
> Application cwnd . “cwnd ” .

---

3 (Costs of Congestion)

TCP , .

3.1 Scenario 1: (,)

: (A, B) R . , .

, (λ_{in}) $R/2$ (delay) .

$$\lim_{\lambda_{in} \to R/2} \text{delay} = \infty$$

delay ?

. μ, λ :

$$E[\text{delay}] \propto \frac{1}{\mu - \lambda}$$

R $R/2$. sender $\lambda_{in} \to R/2$ R .

> **Tip**
>
> **:** . .
>
> | λ_{in} | delay |
> |---|---|
> | $\ll R/2$ | , |
> | $\to R/2$ ( ) | $\to \infty$ |

: throughput $R/2$, delay . “ ” .

3.2 Scenario 2: + ( )

: , .

:

(Perfect Knowledge): sender → , goodput = offered load.

(Known Loss): goodput $R/2$ overhead .

$$\lambda_{out} < \lambda'_{in} \quad (\lambda'_{in} = \text{original} + \text{retransmitted})$$

: **(Premature Timeout):** sender ( ) → → goodput $R/4$.

> **Warning**
>
> , **(positive feedback)** .
> **(Scenario 2):** 1. goodput 2. **goodput** $(\lambda_{out} < \lambda_{in})$

3.3 Scenario 3: ( )

: 4 , 2-hop , .

```
Host A  → [R1]  → [R2]  → Host C
                  ↑
Host D        → Host B
```

A→C R1, R2 . D→B R2 .

- R2 A→C D→B
- D→B R2 A→C
- **R1 A→C , R2**

> **Warning**
>
> **Scenario 3 :**
> "upstream , downstream ."
> λ'_{in} (A→C) D→B goodput 0 . .

:

	Scenario 1	Scenario 2	Scenario 3
goodput	delay → ∞ $R/2$ ( )	goodput $< R/2$ ( )	→ 0 ( )

4 TCP

TCP (state) .

4.1 Slow Start (SS)

`cwnd` **(exponential)** .

:

- : $cwnd = 1$ MSS
- **ACK** : $cwnd \mathrel{+}= 1$ MSS
- : RTT `cwnd` **2**

$$cwnd(t + 1 \text{ RTT}) = 2 \cdot cwnd(t)$$

(exponential) ?

“ **1** ” . RTT 1 **ACK 1** **1** .

$cwnd = N$ MSS RTT N , ACK N . ACK `cwnd += 1 MSS` :

$$cwnd_{\text{RTT}} = cwnd + \underbrace{N}_{\text{ACK}} \times 1 \text{ MSS} = cwnd + cwnd = 2 \times cwnd$$

RTT **2** . $1 \to 2 \to 4 \to 8 \to 16 \to \cdots$ — .

> **Note**
>
> “RTT 1 ” , **“ACK 1 ” → “RTT cwnd ”** .

:

`cwnd >= ssthresh`	Congestion Avoidance
Timeout ( )	`ssthresh = cwnd/2`, `cwnd = 1 MSS`, SS
3 Duplicate ACKs	`ssthresh = cwnd/2`, `cwnd = ssthresh + 3`, Fast Recovery

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

def simulate_tcp(initial_ssthresh=16, max_rounds=25, loss_at=None):
    cwnd = 1
    ssthresh = initial_ssthresh
    cwnd_history = [cwnd]
    ssthresh_history = [ssthresh]
    states = ['SS']
    for rtt in range(1, max_rounds):
        if loss_at and rtt in loss_at:
            ssthresh = max(cwnd // 2, 1)
            if loss_at[rtt] == 'timeout':
                cwnd = 1
                states.append('SS')
            else:
                cwnd = ssthresh + 3
                states.append('FR')
        else:
            if cwnd < ssthresh:
                cwnd *= 2
                states.append('SS')
            else:
                cwnd += 1
                states.append('CA')
        cwnd_history.append(cwnd)
        ssthresh_history.append(ssthresh)
    return cwnd_history, ssthresh_history, states

cwnd_h, ssthresh_h, states = simulate_tcp(
    initial_ssthresh=16, max_rounds=25,
    loss_at={8: '3dup', 17: 'timeout'}
)

fig, ax = plt.subplots(figsize=(12, 6))
colors = {'SS': '#2196F3', 'CA': '#4CAF50', 'FR': '#FF9800'}
state_colors = [colors.get(s, '#9E9E9E') for s in states]

for i in range(len(cwnd_h)-1):
    ax.plot([i, i+1], [cwnd_h[i], cwnd_h[i+1]],
            color=state_colors[i+1], linewidth=2.5)
    ax.scatter(i, cwnd_h[i], color=state_colors[i], s=80, zorder=5)
ax.scatter(len(cwnd_h)-1, cwnd_h[-1], color=state_colors[-1], s=80, zorder=5)

ax.step(range(len(ssthresh_h)), ssthresh_h, where='post',
        linestyle='--', color='red', alpha=0.7, linewidth=1.5)
```

```
ax.axvline(x=8,  color='orange', linestyle=':', alpha=0.8, linewidth=1.5)
ax.axvline(x=17, color='purple', linestyle=':', alpha=0.8, linewidth=1.5)
ax.text(8.2,  max(cwnd_h)*0.85, '3 Dup ACK\n→ Fast Recovery', fontsize=9, color='darkorange')
ax.text(17.2, max(cwnd_h)*0.85, 'Timeout\n→ SS restart',      fontsize=9, color='purple')

ax.set_xlabel('Transmission Round (RTT)', fontsize=12)
ax.set_ylabel('Congestion Window (MSS)', fontsize=12)
ax.set_title('TCP Reno: cwnd Evolution\n(RTT=8: 3 Dup ACK → Fast Recovery  |  RTT=17: Timeout →
ax.grid(True, alpha=0.3)

patches = [
    mpatches.Patch(color='#2196F3', label='Slow Start (SS): ACK  +1MSS → RTT  2 '),
    mpatches.Patch(color='#4CAF50', label='Congestion Avoidance (CA): RTT  +1MSS'),
    mpatches.Patch(color='#FF9800', label='Fast Recovery (FR)'),
    mpatches.Patch(color='red',     label='ssthresh'),
]
ax.legend(handles=patches, loc='upper left', fontsize=9)
plt.tight_layout()
plt.show()
```

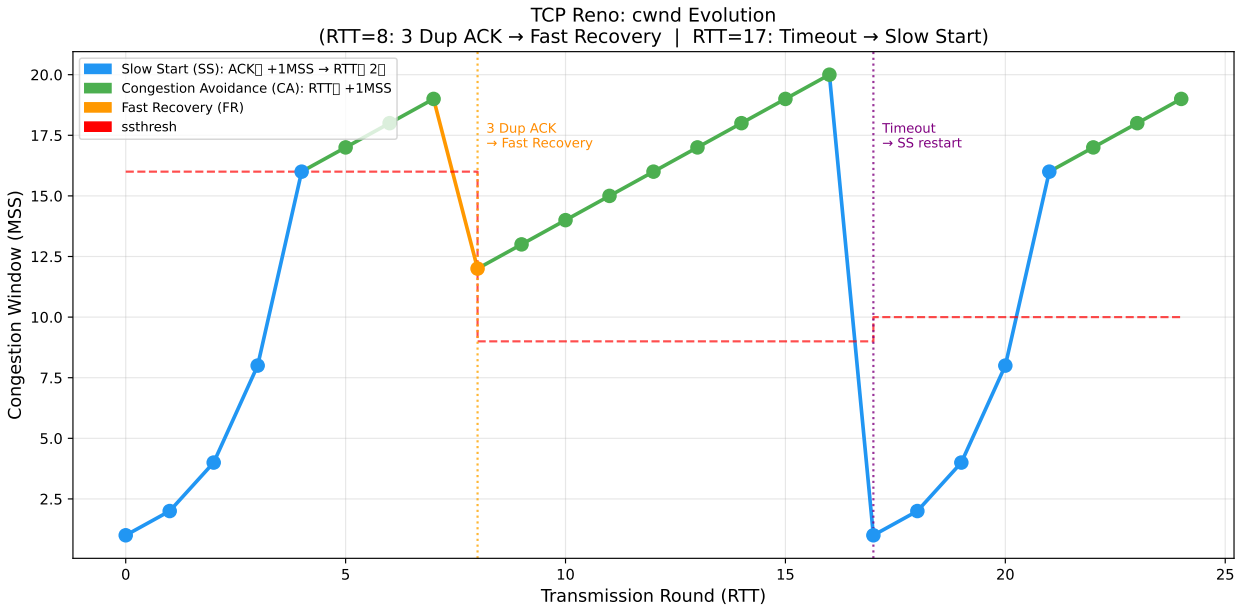


Figure 1: TCP Reno: cwnd (SS→CA→FR→SS)

4.2 Congestion Avoidance (CA)

`cwnd >= ssthresh` , `cwnd` **(linear)** .

:

- ACK : $cwnd \mathrel{+}= \frac{MSS^2}{cwnd}$
- : RTT $cwnd$ **+1 MSS** (Additive Increase)
- : $ssthresh = cwnd/2$, $cwnd \leftarrow cwnd/2$ (Multiplicative Decrease)

(linear) ? — `MSS²/cwnd`

CA RTT +1 MSS . ACK :

$$\underbrace{\frac{cwnd}{MSS}}_{\text{RTT ACK}} \times x = \text{MSS} \implies x = \frac{MSS^2}{cwnd}$$

`MSS²/cwnd` "RTT +1 MSS " .

$$\frac{MSS^2}{cwnd} = MSS \times \underbrace{\frac{MSS}{cwnd}}_{\text{cwnd 1}}$$

cwnd ACK . : MSS = 1,460 bytes, cwnd = 14,600 bytes (10 segments) :

$$\frac{1460^2}{14600} = 146 \text{ bytes/ACK}$$

10 ACK $146 \times 10 = 1,460 = \text{MSS} \rightarrow$ RTT +1 MSS .

SS CA :

	Slow Start	Congestion Avoidance
ACK 1	+1 MSS	$+MSS^2/cwnd$ ( )
RTT 1	$+cwnd$ ( ) (exponential)	+1 MSS ( ) (linear)

CA , ssthresh " " .

AIMD (Additive Increase, Multiplicative Decrease) .

AIMD: $\underbrace{cwnd \mathrel{+}= 1 \text{ MSS per RTT}}_{\text{Additive Increase}}$, $\underbrace{cwnd \leftarrow cwnd/2 \text{ on loss}}_{\text{Multiplicative Decrease}}$

Tip

AIMD

, AIMD throughput **45° (equal share line)** .

- **AI** : $(+1, +1)$ — $(45°)$
- **MD** : —

throughput ( ) ( ) .

4.3 Fast Retransmit & Fast Recovery (FR)

Fast Retransmit: 3 Duplicate ACK , . Dup ACK “ ” , timeout .

Fast Recovery (TCP Reno): Fast Retransmit Slow Start , `ssthresh` Congestion Avoidance .

```
# Fast Recovery     : 3 Dup ACKs
ssthresh = cwnd / 2
cwnd     = ssthresh + 3   # +3:      3  Dup ACK
→ Fast Recovery
  -    Dup ACK   : cwnd += 1 MSS
  -   ACK    :    cwnd = ssthresh  → CA
  - Timeout    :   cwnd = 1 MSS    → SS
```

> **Note**
>
> `+3` : 3 Dup ACK “3 ” . in-flight .

---

5 TCP Tahoe vs. TCP Reno

	TCP Tahoe	TCP Reno
Timeout	`ssthresh=cwnd/2`, `cwnd=1`, SS	
3 Dup ACK	`ssthresh=cwnd/2`, **`cwnd=1`**, SS	`ssthresh=cwnd/2`, **`cwnd=ssthresh+3`**, Fast Recovery
	3 Dup ACK = Timeout	3 Dup ACK = ( )

> **Note**
>
> **TCP Reno Tahoe**
> 3 Dup ACK . cwnd 1 MSS . Reno Fast Recovery throughput . Reno Tahoe throughput **loss** .

```
import matplotlib.pyplot as plt

rounds     = list(range(0, 20))
cwnd_tahoe = [1, 2, 4, 8, 9, 10, 11, 12, 1, 2, 4, 6, 8, 9, 10, 11, 12, 13, 14, 15]
cwnd_reno  = [1, 2, 4, 8, 9, 10, 11, 12, 9, 6, 7, 8,  9, 10, 11, 12, 13, 14, 15, 16]
ssthresh   = [8]*8 + [6]*12

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(rounds, cwnd_tahoe, 'b-o', markersize=6, linewidth=2, label='TCP Tahoe')
ax.plot(rounds, cwnd_reno,  'g-s', markersize=6, linewidth=2, label='TCP Reno')
ax.step(range(20), ssthresh, 'r--', where='post', linewidth=1.5, alpha=0.8, label='ssthresh')
ax.axvline(x=8, color='orange', linestyle=':', linewidth=2, label='3 Dup ACK @ round 8 (cwnd=12

# Reno
ax.fill_between(range(8, 14),
                cwnd_tahoe[8:14], cwnd_reno[8:14],
                alpha=0.15, color='green', label='Reno    ')

ax.annotate('Tahoe: cwnd→1\n(SS restart)',
            xy=(8, 1), xytext=(9.5, 2.5),
            arrowprops=dict(arrowstyle='->', color='blue'), fontsize=9, color='blue')
ax.annotate('Reno: cwnd→ssthresh+3=9\n(Fast Recovery  )',
            xy=(8, 9), xytext=(9.5, 11),
            arrowprops=dict(arrowstyle='->', color='green'), fontsize=9, color='green')

ax.set_xlabel('Transmission Round', fontsize=12)
ax.set_ylabel('cwnd (MSS)', fontsize=12)
ax.set_title('TCP Tahoe vs. Reno: cwnd Evolution\n(3 Dup ACKs at Round 8, cwnd=12 → ssthresh=6)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
ax.set_ylim(0, 20)
plt.tight_layout()
plt.show()
```

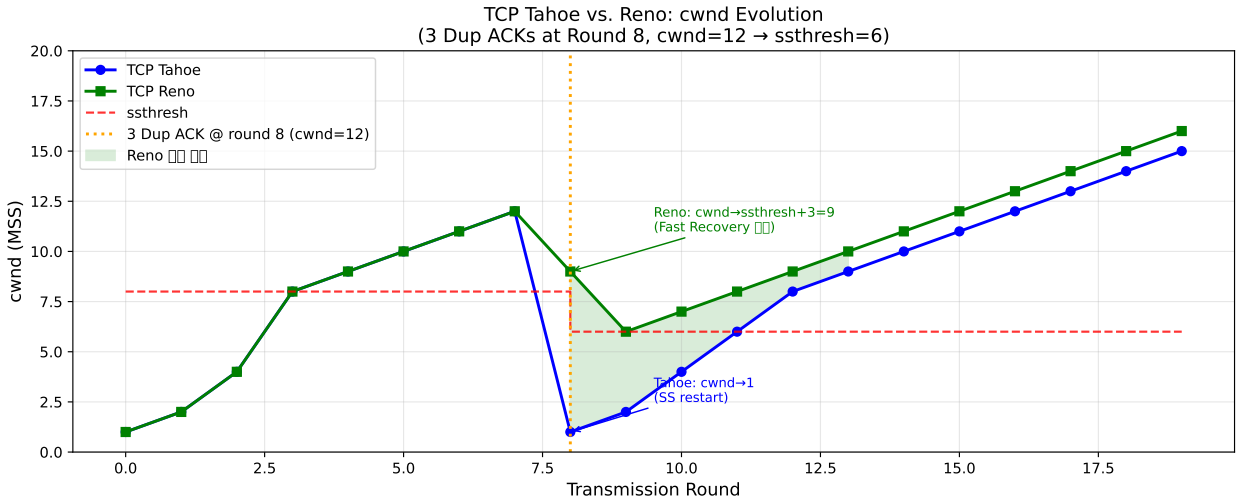


Figure 2: TCP Tahoe vs. Reno: 3 Dup ACK cwnd (ssthresh=8, loss at cwnd=12)

---

6 TCP FSM

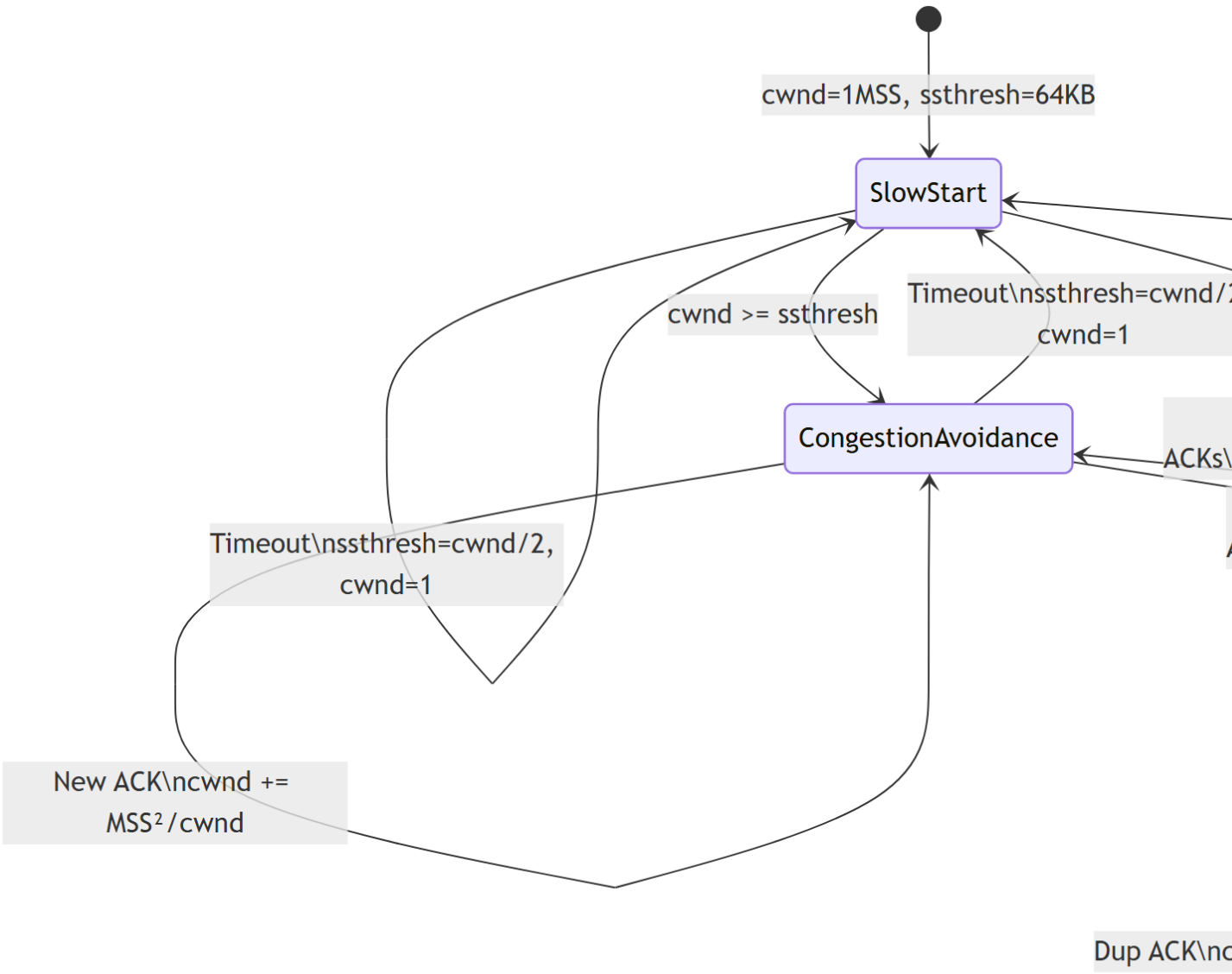
TCP Reno Finite State Machine .

```
stateDiagram-v2
    [*] --> SlowStart : cwnd=1MSS, ssthresh=64KB

    SlowStart --> SlowStart : New ACK\ncwnd += MSS
    SlowStart --> CongestionAvoidance : cwnd >= ssthresh
    SlowStart --> SlowStart : Timeout\nssthresh=cwnd/2, cwnd=1
    SlowStart --> FastRecovery : 3 Dup ACKs\nssthresh=cwnd/2\ncwnd=ssthresh+3

    CongestionAvoidance --> CongestionAvoidance : New ACK\ncwnd += MSS²/cwnd
    CongestionAvoidance --> SlowStart : Timeout\nssthresh=cwnd/2, cwnd=1
    CongestionAvoidance --> FastRecovery : 3 Dup ACKs\nssthresh=cwnd/2\ncwnd=ssthresh+3

    FastRecovery --> FastRecovery : Dup ACK\ncwnd += MSS
    FastRecovery --> CongestionAvoidance : New ACK\ncwnd=ssthresh
    FastRecovery --> SlowStart : Timeout\nssthresh=cwnd/2, cwnd=1
```



---

7 TCP Throughput

7.1 (AIMD)

AIMD `cwnd` $W/2$ W . `cwnd = W` :

$$\overline{\text{throughput}} = \frac{3}{4} \cdot \frac{W}{RTT} \quad \text{[bytes/sec]}$$

: $= \frac{W/2 + W}{2} = \frac{3W}{4}$

7.2 Mathis Formula ( )

L [Mathis 1997]:

$$\overline{\text{throughput}} = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$$

> **Tip**
>
> ( )
> AIMD :
>
> - Additive Increase: $W/2$ W $W/2$ RTT
> - : $\sum_{k=W/2}^{W} k \cdot MSS \approx \frac{3W^2}{8} \cdot MSS$
> - $1 \rightarrow L = \frac{8}{3W^2}$
> - $W = \sqrt{\frac{8}{3L}}$ throughput $\frac{1.22 \cdot MSS}{RTT\sqrt{L}}$

7.3 Long Fat Pipe

(: 10 Gbps, RTT=100ms) :

$$L \leq 2 \times 10^{-10}$$

. CUBIC, BBR TCP .

```
import numpy as np
import matplotlib.pyplot as plt

MSS = 1500
RTT = 0.1
L_range = np.logspace(-10, -2, 500)
throughput_Mbps = (1.22 * MSS) / (RTT * np.sqrt(L_range)) / 1e6

fig, ax = plt.subplots(figsize=(10, 5))
ax.loglog(L_range, throughput_Mbps, 'b-', linewidth=2.5, label='Mathis Formula')

thresholds = {
    10000: ('10 Gbps', 'red'),
    1000:  ('1 Gbps',  'orange'),
```

```
    100:   ('100 Mbps','green'),
    10:    ('10 Mbps', 'steelblue'),
}
for tp, (label, color) in thresholds.items():
    L_needed = (1.22 * MSS / (RTT * tp * 1e6))**2
    ax.axhline(y=tp, color=color, linestyle='--', alpha=0.5, linewidth=1)
    ax.axvline(x=L_needed, color=color, linestyle=':', alpha=0.5, linewidth=1)
    ax.text(L_needed * 1.5, tp * 1.4,
            f'{label}\nL={L_needed:.1e}', fontsize=8, color=color)

ax.set_xlabel('Packet Loss Rate (L)', fontsize=12)
ax.set_ylabel('Throughput (Mbps)', fontsize=12)
ax.set_title('TCP Throughput vs. Loss Rate\n(Mathis Formula, MSS=1500B, RTT=100ms)', fontsize=
ax.grid(True, which='both', alpha=0.3)
ax.legend()
plt.tight_layout()
plt.show()
```

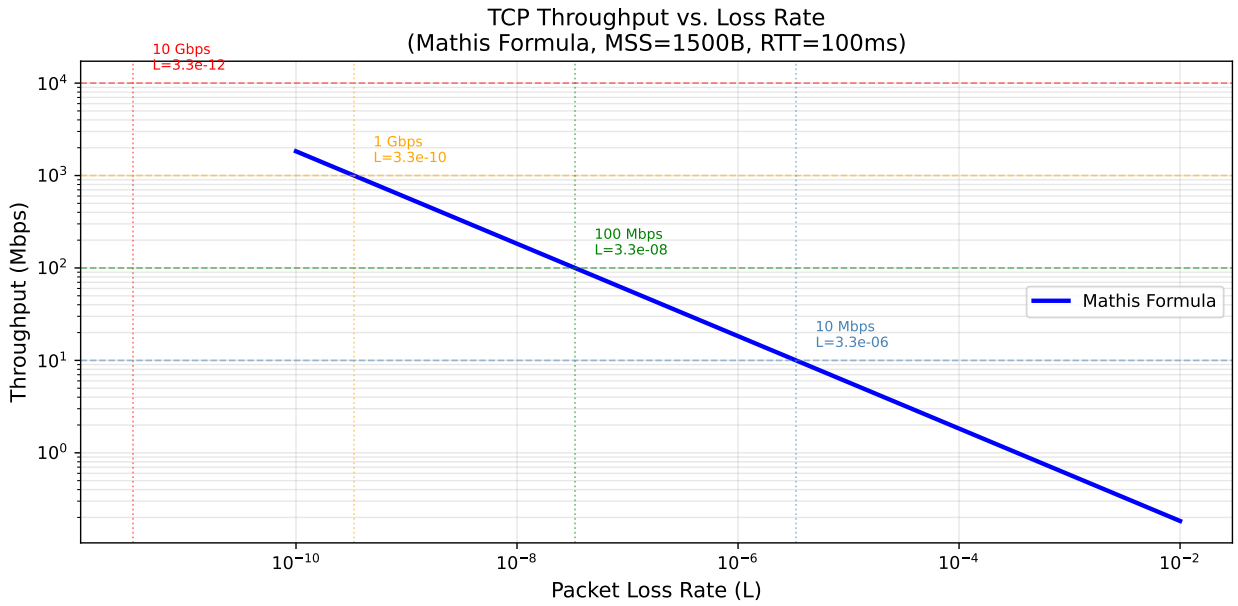


Figure 3: Mathis Formula: TCP Throughput (MSS=1500B, RTT=100ms)

---

8 TCP (Fairness)

8.1 AIMD

K TCP R , :

$$= \frac{R}{K}$$

:

- **AI** : $(+1, +1)$ — $(45°)$
- **MD** : $(0.5 \cdot x_1, 0.5 \cdot x_2)$ —

throughput ( ) ( ) .

```
import numpy as np
import matplotlib.pyplot as plt

R = 10
fig, ax = plt.subplots(figsize=(8, 8))

ax.fill_between([0, R], [R, 0], [R, R], alpha=0.05, color='red')
ax.fill_between([0, R], [0, 0], [R, 0], alpha=0.05, color='green')
ax.plot([0, R], [0, R], 'k--', linewidth=1.5, label='Equal share (45°)')
ax.plot([0, R], [R, 0], 'r-',  linewidth=2,   alpha=0.7, label='Capacity limit')

np.random.seed(42)
x1, x2 = 1.0, 7.0
trajectory = [(x1, x2)]
for _ in range(20):
    x1 += 1; x2 += 1
    if x1 + x2 > R:
        x1 *= 0.5; x2 *= 0.5
    trajectory.append((x1, x2))

xs, ys = zip(*trajectory)
ax.plot(xs, ys, 'b-o', markersize=5, linewidth=1.5, alpha=0.8, label='AIMD trajectory')
ax.scatter(xs[0], ys[0], s=150, c='blue', marker='^', zorder=6, label='Start')
ax.scatter(xs[-1], ys[-1], s=150, c='red',  marker='*', zorder=6, label='Converged')

mid_idx = 5
ax.annotate('', xy=(xs[mid_idx]+1, ys[mid_idx]+1),
            xytext=(xs[mid_idx], ys[mid_idx]),
            arrowprops=dict(arrowstyle='->', color='green', lw=2))
ax.text(xs[mid_idx]+0.3, ys[mid_idx]+0.2, 'AI (+1,+1)', fontsize=9, color='green')

ax.annotate('', xy=(xs[mid_idx+1]*0.5, ys[mid_idx+1]*0.5),
            xytext=(xs[mid_idx+1], ys[mid_idx+1]),
            arrowprops=dict(arrowstyle='->', color='red', lw=2))
ax.text(xs[mid_idx+1]*0.5+0.2, ys[mid_idx+1]*0.5+0.3, 'MD (×0.5)', fontsize=9, color='red')

ax.set_xlim(0, R+1); ax.set_ylim(0, R+1)
ax.set_xlabel('Throughput of Connection 1', fontsize=12)
```

```
ax.set_ylabel('Throughput of Connection 2', fontsize=12)
ax.set_title('AIMD Fairness Convergence\n(Two TCP Connections, Link Capacity R=10)', fontsize=
ax.legend(fontsize=10, loc='upper right')
ax.grid(True, alpha=0.3)
ax.set_aspect('equal')
plt.tight_layout()
plt.show()
```

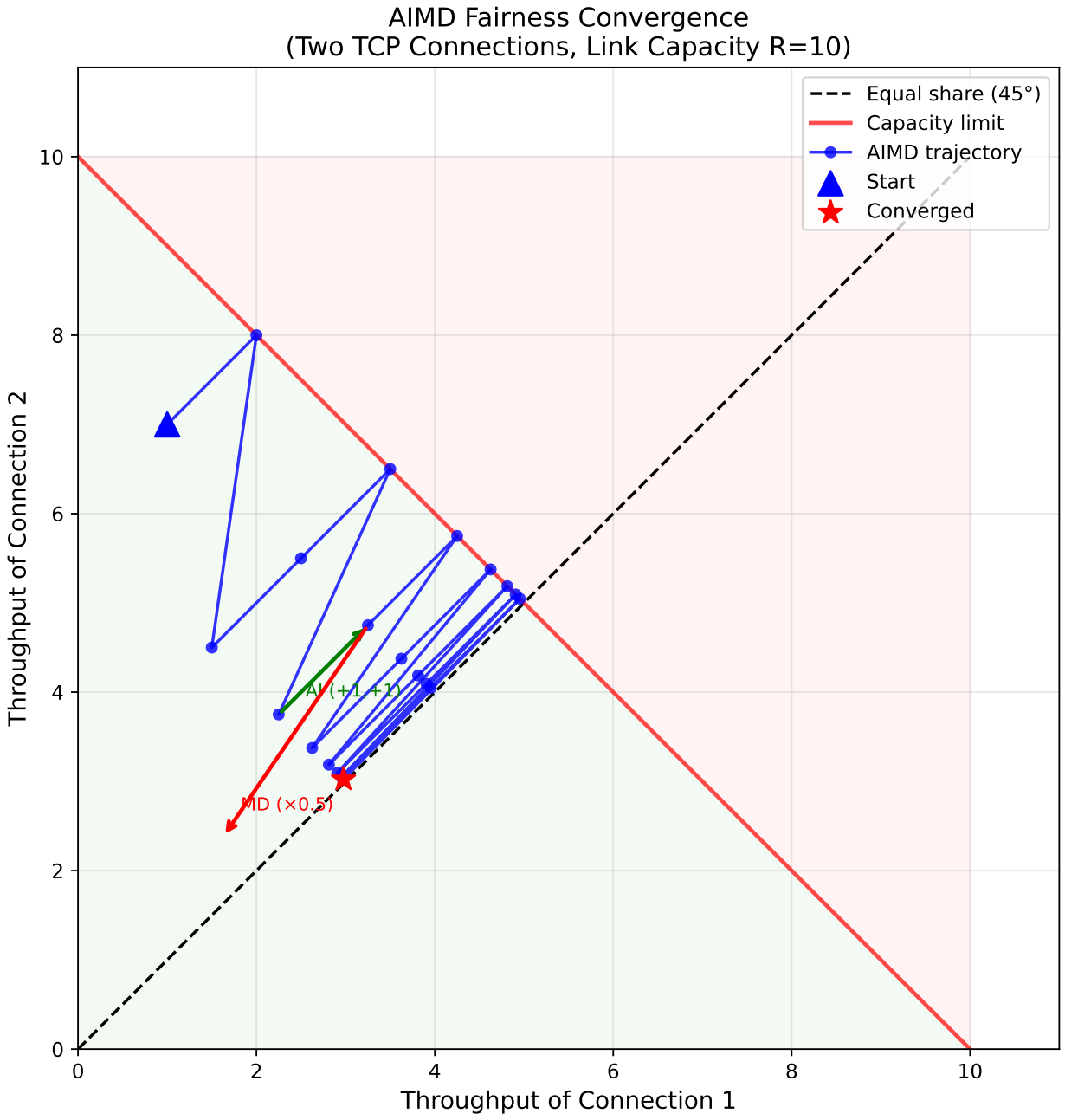


Figure 4: AIMD Fairness: TCP Throughput

8.2

- **RTT** : RTT `cwnd` → RTT throughput
- **TCP** : N N
- **UDP** : UDP TCP

---

9 Explicit Congestion Notification (ECN)

TCP **(loss)** , ECN (network-assisted) .

9.1

```
[ ]                → [ ]
          IP datagram: ECN=10 (capable)
                   ↓ (       )
          IP datagram: ECN=11 (CE: Congestion Experienced)
[ ] ←                [ ]
          TCP ACK: ECE=1 (Echo CE   )
[ ]: cwnd      , CWR=1
```

ECN	
`00`	ECN
`10` or `01`	ECN (ECT: ECN-Capable Transport)
`11`	(CE: Congestion Experienced)

> **Note**
>
> **ECN** : , . (DCTCP) .

10

10.1

---

rate $\approx cwnd/RTT$

throughput	$\overline{T} = \frac{3}{4} \cdot \frac{W}{RTT}$
Mathis throughput	$\overline{T} = \frac{1.22 \cdot MSS}{RTT \cdot \sqrt{L}}$
CA ACK	$\Delta cwnd = MSS^2/cwnd$ $= R/K$

10.2 cwnd

		cwnd	
Slow Start	New ACK	`cwnd += 1 MSS`	ACK → RTT 2
Slow Start	Timeout	`ssthresh=cwnd/2, cwnd=1`	
Slow Start	3 Dup ACK	`ssthresh=cwnd/2,` `cwnd=ssthresh+3` → FR	Reno
Congestion Avoidance	New ACK	`cwnd += MSS²/cwnd`	RTT +1 MSS
Congestion Avoidance	Timeout	`ssthresh=cwnd/2, cwnd=1`	
Congestion Avoidance	3 Dup ACK	`ssthresh=cwnd/2,` `cwnd=ssthresh+3` → FR	
Fast Recovery	Dup ACK	`cwnd += 1 MSS`	
Fast Recovery	New ACK	`cwnd=ssthresh` → CA	
Fast Recovery	Timeout	`ssthresh=cwnd/2, cwnd=1` → SS	

10.3

`:ssthresh = 8 MSS`, `cwnd = 1 MSS` TCP Reno .
3 Dup ACK , `cwnd = 12 MSS`.

1. `ssthresh` ?
2. Fast Recovery `cwnd` ?
3. ACK `cwnd` ?

---

cwnd ( ):

Round		cwnd	
1	SS	1	
2	SS	2	ACK +1 → RTT 2
3	SS	4	

Round		cwnd	
4	SS→CA	8 = ssthresh → CA	cwnd ssthresh
5	CA	9	RTT +1
6	CA	10	
7	CA	11	
8	CA	**12**	**→ 3 Dup ACK**

:

Q1. ssthresh:

$$ssthresh = \frac{cwnd}{2} = \frac{12}{2} = \boxed{6 \text{ MSS}}$$

Q2. Fast Recovery cwnd (TCP Reno):

$$cwnd = ssthresh + 3 = 6 + 3 = \boxed{9 \text{ MSS}}$$

Q3. Missing segment ACK :

$$cwnd = ssthresh = \boxed{6 \text{ MSS}}, \quad \rightarrow \text{Congestion Avoidance}$$

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

rounds = list(range(0, 16))
cwnd   = [1, 2, 4, 8, 9, 10, 11, 12, 9, 6, 7, 8, 9, 10, 11, 12]
sst    = [8]*8 + [6]*8
states = ['SS','SS','SS','SS','CA','CA','CA','CA','FR','CA','CA','CA','CA','CA','CA','CA']

colors = {'SS':'#2196F3', 'CA':'#4CAF50', 'FR':'#FF9800'}
fig, ax = plt.subplots(figsize=(12, 5))

for i in range(len(cwnd)-1):
    ax.plot([rounds[i], rounds[i+1]], [cwnd[i], cwnd[i+1]],
            color=colors[states[i+1]], linewidth=2.5)
    ax.scatter(rounds[i], cwnd[i], color=colors[states[i]], s=70, zorder=5)
ax.scatter(rounds[-1], cwnd[-1], color=colors[states[-1]], s=70, zorder=5)

ax.step(rounds, sst, 'r--', where='post', linewidth=1.5, alpha=0.8)

ax.axvline(x=7, color='orange', linestyle=':', linewidth=2)
ax.annotate('3 Dup ACK\ncwnd=12\n→ ssthresh=6, cwnd=9 (FR)',
            xy=(7, 12), xytext=(8.5, 13),
            arrowprops=dict(arrowstyle='->', color='darkorange'),
            fontsize=9, color='darkorange',
```

```
            bbox=dict(boxstyle='round,pad=0.3', facecolor='lightyellow', alpha=0.8))

ax.annotate('New ACK\ncwnd = ssthresh = 6\n→ CA  ',
            xy=(9, 6), xytext=(10.5, 3.5),
            arrowprops=dict(arrowstyle='->', color='green'),
            fontsize=9, color='darkgreen')

ax.set_xlabel('Transmission Round', fontsize=12)
ax.set_ylabel('cwnd (MSS)', fontsize=12)
ax.set_title('   : TCP Reno cwnd  \n(ssthresh=8, 3 Dup ACK at cwnd=12 → ssthresh=6, FR     CA
ax.grid(True, alpha=0.3)
ax.set_ylim(0, 16)

patches = [
    mpatches.Patch(color='#2196F3', label='Slow Start (SS)'),
    mpatches.Patch(color='#4CAF50', label='Congestion Avoidance (CA)'),
    mpatches.Patch(color='#FF9800', label='Fast Recovery (FR)'),
    mpatches.Patch(color='red',     label='ssthresh'),
]
ax.legend(handles=patches, loc='upper left', fontsize=9)
plt.tight_layout()
plt.show()
```

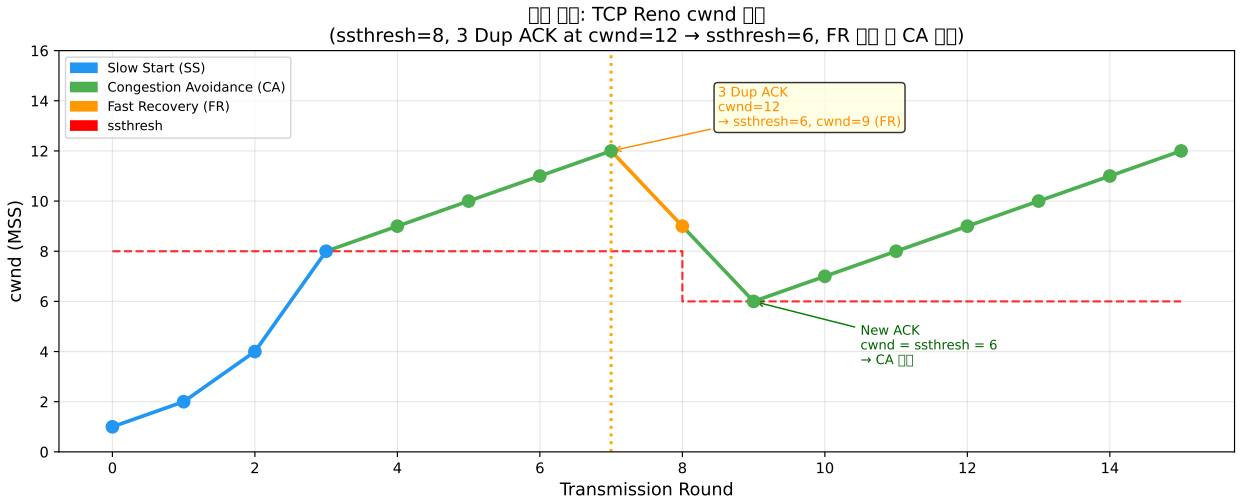


Figure 5: : ssthresh=8, 3 Dup ACK at cwnd=12

11

- Kurose, J. F. & Ross, K. W. (2016). *Computer Networking: A Top-Down Approach*, 7th Edition. Pearson.

  - Chapter 3.6: Principles of Congestion Control
  - Chapter 3.7: TCP Congestion Control

- Jacobson, V. (1988). Congestion Avoidance and Control. *ACM SIGCOMM*.
- Mathis, M. et al. (1997). The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm. *ACM SIGCOMM Computer Communication Review*.
- RFC 5681 — TCP Congestion Control.
- RFC 3168 — The Addition of Explicit Congestion Notification (ECN) to IP.